

Pygame Simon game sequence bug

09/30/2026 21:34:24

<https://claude.ai/chat/63876efa-db9b-4738-a807-f75b95eeccf2>

I'm working on a Pygame Simon-style memory game. Each round should...

```
```python self.sequence.append(new_color) ```
```

give the modified function

```
```python def start_next_round(self): new_color = random.randint(0, 3)...
```

Currently, the sequence flashes at a static 450ms flash duration and...

```
```python def start_next_round(self): new_color = random.randint(0, 3)...
```

Here is my Pygame memory game code The game currently relies entire...

I'll add pure-Python tone synthesis to the engine and hook it into both...

ok now for task 4: During PLAYER\_TURN, the player can currently wait...

I added a per-step timer: the player gets 3 seconds for each click, and...

ok give complete game engine updated for all tasks

I've attached `game\_engine.py` with all four tasks combined: 1....

so give main file as well

I've attached the updated `main.py`. The only change is one new line ju...



game\_engine.py

I'm working on a Pygame Simon-style memory game. Each round should add exactly one new random color to the sequence, but the sequence grows exponentially like 1,3,7,15 instead of 1,2,3,4. Find the bug and give me only the corrected line

9/30/2026, 8:34:44 AM



```
self.sequence.append(new_color)
```

9/30/2026, 8:34:46 AM



give the modified function

9/30/2026, 8:38:39 AM



```
def start_next_round(self):
 new_color = random.randint(0, 3)
 self.sequence.append(new_color)

 self.player_input.clear()
 self.state = "WATCH"
 self.showing_step = 0
 self.step_start_time = pygame.time.get_ticks()
 self.is_flashing = True
 self.buttons[self.sequence[0]].is_lit = True
```

9/30/2026, 8:38:41 AM



Currently, the sequence flashes at a static 450ms flash duration and 200ms pause duration across all rounds. In `game_engine.start_next_round()`, adjust `self.flash_duration` and `self.pause_duration` to decrease gradually as `self.score` increases (down to a minimum speed threshold like 180ms flash and 80ms pause), making playback faster and more challenging in later rounds.

9/30/2026, 8:42:15 AM



```
def start_next_round(self):
 new_color = random.randint(0, 3)
 self.sequence.append(new_color)

 # Speed up playback as score rises, clamped to minimum thresholds
 self.flash_duration = max(180, 450 - self.score * 25)
 self.pause_duration = max(80, 200 - self.score * 10)

 self.player_input.clear()
 self.state = "WATCH"
 self.showing_step = 0
 self.step_start_time = pygame.time.get_ticks()
 self.is_flashing = True
 self.buttons[self.sequence[0]].is_lit = True
```

Flash hits its 180ms floor at score 11, and pause hits 80ms at score 12. Since `reset()` sets the score to 0 before calling this, speeds go back to 450/200 automatically on restart. Adjust the step sizes (25 and 10) if you want a gentler or steeper ramp.

9/30/2026, 8:42:20 AM



game\_engine.py

main.py

Here is my Pygame memory game code

The game currently relies entirely on silent visual flashes. Use `pygame.mixer` or synthesize simple square/sine audio tones mapped to each color ID (e.g., Red: 261Hz, Blue: 329Hz, Green: 392Hz, Yellow: 523Hz). Play the respective tone whenever a button flashes during WATCH mode and when the player clicks it during PLAYER\_TURN.

dont use numpy and no sound files

9/30/2026, 8:57:17 PM



Tuning startup latency and stopping sounds on reset. ▾

- 🕒 Tuning startup latency and stopping sounds on reset.
- 🕒 Done

I'll add pure-Python tone synthesis to the engine and hook it into both playback and clicks.

- 🔑 Ran a command
- 🔑 Ran a command
- 🔑 Ran a command
- 🔑 Shared files
- 🕒 Done

Each color now has its own tone, built in pure Python with the `array` and `math` modules and played through `pygame.mixer.Sound(buffer=...)`. It uses no numpy and no sound files. I ran it with a dummy audio driver: the mixer started, and a 450 ms tone took about 9 ms to build and came out exactly 450 ms long. I couldn't actually listen to it here, so check the sound on your machine.

What changed:

- `main.py`: I added one line before `pygame.init()`: `pygame.mixer.pre_init(44100, -16, 1, 512)`. This sets 16-bit mono with a small buffer, so the sound plays with

little delay after the flash.

- `game_engine.py`:
  - A `TONE_FREQS` table: Red 261.63 Hz, Blue 329.63 Hz, Green 392 Hz, Yellow 523.25 Hz.
  - `_make_tone()` builds each tone and fades it in and out over 10 ms so it doesn't click. Each tone is saved after it's first made.
  - `play_tone(color_id, duration_ms)` plays a tone. It's called in three places:
    - In `start_next_round()`, for the first flash.
    - In `update()`, each time the next button lights during WATCH.
    - In `handle_event()`, when the player clicks a button during PLAYER\_TURN.
- **Tone length:**
  - During WATCH, each tone lasts exactly as long as the flash (`self.flash_duration`). As rounds get faster, the tones get shorter to match.
  - When the player clicks, the tone lasts 220 ms (`self.player_tone_duration`).
- **No audio device:** if the mixer can't start, the game still runs without sound instead of crashing.

To switch to a buzzer, retro sound, set `self.waveform = "square"`. The square wave is played at a lower volume because it sounds much louder than a sine wave.

9/30/2026, 8:58:17 PM



ok now for task 4: During PLAYER\_TURN, the player can currently wait indefinitely before making their move. Add a per-round or per-step countdown timer bar displayed in the HUD. If the timer reaches 0 before the player completes the required sequence, transition the game state to GAME\_OVER. give just changed code in game\_engine

9/30/2026, 9:08:05 PM



- ⌚ Deciding on timeout rules and placing the timer bar in the HUD.
- 🔑 Ran a command
- ⌚ Rebuilding a game from memory using audio snippets.
- 🔑 Ran a command
- 🔑 Ran a command
- ✅ Done

I added a per-step timer: the player gets 3 seconds for each click, and every correct click refills the bar. I tested it without a screen or sound: a correct click started the next round, and waiting ran out at about 3.0 s and switched to **GAME\_OVER** with the timeout message. These edits don't touch the sound code, so they sit fine alongside the tone changes.

1. **\_\_init\_\_**: add after **self.player\_flash\_duration = 150**

```
Per-step countdown during PLAYER_TURN
self.step_time_limit = 3000 # ms allowed for each click
self.turn_start_time = 0
self.game_over_reason = ""
```

2. **update()**: replace the final **else:** block in WATCH, and add a PLAYER\_TURN branch

```
else:
 self.state = "PLAYER_TURN"
 self.turn_start_time = now
```

```

elif self.state == "PLAYER_TURN":
 if now - self.turn_start_time >= self.step_time_limit:
 self.game_over_reason = "TIME'S UP!"
 self.state = "GAME_OVER"

```

### 3. `register_player_click()`

```

def register_player_click(self, color_id):
 self.player_input.append(color_id)
 current_idx = len(self.player_input) - 1

 if self.player_input[current_idx] !=
self.sequence[current_idx]:
 self.game_over_reason = "WRONG PATTERN!"
 self.state = "GAME_OVER"
 return

 if len(self.player_input) == len(self.sequence):
 self.score += 1
 self.start_next_round()
 else:
 self.turn_start_time = pygame.time.get_ticks() # correct
 click: refill timer

```

### 4. `render()`: draw the bar before the buttons, and use the reason in the game-over title

```

if self.state == "PLAYER_TURN":
 self.render_timer_bar(screen)

for btn in self.buttons:
 btn.render(screen)

```

```

over_surf = self.font_title.render(f"
{self.game_over_reason} GAME OVER", True, (240, 70, 70))

```

**5. New method:** the bar sits between the status text and the pads. It turns from green to amber to red as time runs out.

```
def render_timer_bar(self, screen):
 elapsed = pygame.time.get_ticks() - self.turn_start_time
 ratio = max(0.0, 1.0 - elapsed / self.step_time_limit)

 bar_w, bar_h = 300, 10
 x = self.width // 2 - bar_w // 2
 y = 124

 if ratio > 0.5:
 color = (80, 240, 130)
 elif ratio > 0.25:
 color = (255, 200, 60)
 else:
 color = (240, 70, 70)

 pygame.draw.rect(screen, (50, 54, 64), (x, y, bar_w, bar_h),
border_radius=5)
 if ratio > 0:
 pygame.draw.rect(screen, color, (x, y, int(bar_w *
ratio), bar_h), border_radius=5)
```

If you'd rather have one timer for the whole round, delete the `else:` refill in step 3. Then set `self.step_time_limit = 2000 + len(self.sequence) * 800` in `start_next_round()`, so longer sequences get more time.

9/30/2026, 9:09:01 PM



ok give complete game engine updated for all tasks

9/30/2026, 9:12:06 PM



- 🔑 Ran a command
- 🔑 Shared files
- 🕒 Done

I've attached `game_engine.py` with all four tasks combined:

1. **Sequence bug:** each round now adds exactly one color with `self.sequence.append(new_color)`.
2. **Speed-up:** the flash is 450 ms and the pause 200 ms at the start. Each point of score makes them shorter, down to a minimum of 180 ms and 80 ms.
3. **Tones:** each color plays its own tone, made in plain Python with no numpy and no sound files. You hear it when a button flashes while you watch and when you click one.
4. **Timer bar:** you get 3 seconds for each click, and a correct click refills the bar. If time runs out, the game ends with "TIME'S UP! GAME OVER". A wrong click still shows "WRONG PATTERN! GAME OVER".

I ran it without a screen or sound, using a stand-in for your `ColorButton`:

- Three rounds were played correctly by simulated clicks, and the sequence grew 2, 3, 4.
- The speeds went 425/190, then 400/180, then 375/170.
- Waiting on a turn ended the game after about 3 seconds.
- Pressing R reset the score, the sequence and the speeds.

For the tones, `main.py` still needs the one line from before, placed ahead of

```
pygame.init(): pygame.mixer.pre_init(44100, -16, 1, 512)
```

9/30/2026, 9:12:39 PM



excerpt\_from\_previous\_claude\_message.txt

so give main file as well

9/30/2026, 9:16:34 PM



🔑 Ran a command

🔑 Shared files

👍 Done

I've attached the updated `main.py`. The only change is one new line just before `pygame.init()`:

```
pygame.mixer.pre_init(44100, -16, 1, 512) # 16-bit mono, small
buffer for low-latency tones
```

It sets up sound in the format the tone code expects, with a small buffer so each tone plays in step with its flash. I checked that the file compiles, but I didn't launch the game with it. Put it in your project root next to the `game/` folder, then run `python main.py`.

9/30/2026, 9:16:47 PM